

PS 26132 — Model Build Specifications

What actually needs ML or CV, what needs plain math instead, and the full build spec for each — for a team building from scratch

Market linkages & price discovery for farmers — Govt. of Maharashtra (MSInS)
Prepared for GlobeX — 5 September 2026

2	3	1
components need real ML/CV	components need only formulas	stretch upgrade path

1 The classification, at a glance

Building everything as a trained model is not more rigorous — it’s often less defensible, since a formula you can explain in one sentence beats a model you can’t justify with the data you actually have. Five capabilities, one honest verdict each.

Capability	Verdict	Why
Price / sale-window forecast	Classical ML, not DL	A real time-series prediction problem, but the deployed Dual-Head GRU already lost to a plain moving average on your own benchmark (54.49% vs. 29.42% MAPE). Gradient-boosted quantile regression is the evidence-backed middle tier — see Section 2.
Sale-window ranking (hold/sell, which market)	Formula, not ML	Few, interpretable factors (price trend, storage cost, transport cost, glut risk); no labelled “correct decision” data exists to train against; a weighted MCDA score is fully explainable. See Section 4.
Buyer/FPO matching	Formula, for now	Cold-start problem — no historical match-outcome labels exist yet to learn from. Rule-based filtering plus a similarity score outperforms an untrained learned ranker. Revisit as ML once real outcomes accumulate. See Section 4.
Trust/reputation score	Formula, on purpose	Must be independently recomputable by anyone from the trade history — that auditability is the entire point, and a trained model’s score can’t be verified the same way. See Section 4.
Quality grading	Classical CV + simple classifier; DL is a funded stretch	Grade genuinely depends on visual attributes text can’t capture, so this is the one place CV earns its place — but there is no public labelled dataset of AGMARK-graded produce photos, so treat this as a data-collection problem first. See Section 3.

2 Spec: price / sale-window forecaster

Why classical ML, not deep learning

This isn't a guess — it matches the published literature on agricultural price forecasting. Recent work comparing tree-based and hybrid models against pure deep-learning approaches (e.g. a TCN-XGBoost hybrid for agricultural market prices, and broader comparisons of linear/tree-based ML against commodity price series) consistently finds gradient-boosted trees competitive with or better than deep sequence models on this kind of data, especially when the historical series per commodity-market pair is not very long — exactly your situation with 2–3 states of mandi history. Deep learning needs volume classical ML doesn't.

Build spec

Element	Specification
Target variable	Next- N -day modal price per (commodity, market) pair, plus the 10th/50th/90th percentile bands (quantile targets), not a single point forecast.
Features	Lagged prices (1/7/14/30-day), rolling mean/std of price and arrivals, day-of-week and day-of-year seasonality encodings, arrival-volume trend, min-max price spread (volatility signal — unique to mandi data, keep it), state/market one-hot or target-encoded identifiers.
Algorithm	Gradient-boosted quantile regression (XGBoost/LightGBM with quantile or pinball-loss objective), one model per quantile (10th/50th/90th) or a native multi-quantile objective if the library supports it.
Baselines to beat	7-day and 30-day moving average; naive last-value carry-forward; simple linear trend. Report all three alongside the model — this is exactly the comparison your own benchmark file already does, keep doing it honestly.
Train/test split	Chronological, never random — train on earlier dates, test on later ones, per commodity-market pair, mirroring the anomaly classifier's existing chronological-split discipline.
Evaluation metric	Pinball loss (quantile loss) for the bands; MAPE and R^2 on the median prediction for headline comparability against the baselines.
What “good” looks like	Beats the moving-average baseline on MAPE <i>and</i> the 80% interval (10th–90th percentile band) actually contains the true price roughly 80% of the time on held-out data — calibration matters as much as point accuracy here, since the product promise is a confidence band, not a number.
Stretch upgrade	A hybrid architecture (tree-based model plus a lightweight sequence model for residuals) only if the median-model baseline is solidly beaten first and time remains — don't reach for this before the simple version is honestly evaluated.

3 Spec: quality grading (computer vision)

The honest starting point: no ready-made labelled dataset exists

Public fruit/vegetable image datasets do exist on Kaggle (fresh-vs-rotten classification, general fruit-type recognition, some defect/quality labels), and there is a real, peer-reviewed literature on computer-vision produce grading using exactly the classical features below. But none of the public datasets are labelled against AGMARK's actual grade categories (moisture, size, foreign-matter thresholds per commodity) — they're closest as a *pretraining or sanity-check* resource, not a drop-in substitute. Budget real time for photographing and labelling your own small sample set; that is the actual bottleneck, not the modelling.

Tiered build plan — start at Tier 1, not Tier 3

Tier	Approach	Why this order
0 (base-line, always keep)	OCR/NER check of the submitted grading certificate against the lot record	Already specified, already reliable, zero training data needed — this is the guaranteed-working fallback if the CV tiers below run out of time.
1 (build this)	Classical image features — size via a reference object in frame, colour histogram compared against AGMARK's published per-commodity thresholds, blob/contour-based defect and blemish counting via OpenCV — feeding a simple classifier (Random Forest or SVM)	Data-efficient: works with as few as 50–150 hand-labelled photos per commodity, fully explainable (“flagged for X% surface blemish area,” not a black-box score), and matches the established computer-vision-for-produce-grading literature rather than inventing an untested approach.
2 (stretch only)	Small transfer-learning CNN (e.g. fine-tuned MobileNet/EfficientNet) trained on your own labelled photos	Only attempt if Tier 1 is working and you can realistically gather several hundred labelled images per class — a CNN trained on too few images will overfit and perform <i>worse</i> than Tier 1, which is a real risk if attempted under time pressure.

Build spec (Tier 1)

Element	Specification
Target variable	Grade category per commodity (e.g. AGMARK Grade A/B/C, or a pass/fail against the declared certificate grade), a multi-class or ordinal classification target.
Features	Estimated size/diameter (calibrated against a reference object of known size in the photo), colour histogram distance from the expected healthy-produce range, percentage of surface area flagged as blemished/dicoloured via contour/blob detection, count of visible defects.

Element	Specification
Algorithm	Random Forest or SVM on the engineered features above — deliberately not a CNN at this tier.
Data plan	Self-photograph 50–150 samples per commodity across the grade categories you intend to support (start with 1–2 commodities, not all of them); optionally pre-train feature-extraction thresholds using a public freshness/defect Kaggle dataset as a sanity check, but do not treat it as ground truth for AGMARK grades.
Train/test split	Stratified by grade class, held-out test images never touched during threshold-tuning.
Evaluation metric	Per-class precision/recall/F1, plus a confusion matrix — report where it actually fails (which grade pairs get confused), don't just report overall accuracy.
What “good” looks like	Clearly beats a trivial baseline (always-predict-the-most-common-grade); more importantly, its mistakes should be between <i>adjacent</i> grades (B misclassified as A or C), not wild misses — that's the difference between “imperfect but usable” and “untrustworthy.”

4 Spec: the three formula-based components

These are specified, not trained — validate them like you would any deterministic system: with test cases and backtests, not train/test splits.

Sale-window ranking (MCDA)

Inputs: price-trend score, storage-cost estimate, transport-cost estimate, arrival-volume/glut-risk score — one row per candidate action (sell now at market X , hold N days). **Logic:** weighted sum (or TOPSIS-style normalised distance) across the four dimensions; weights set from domain judgement initially, refined by backtesting against historical price outcomes once enough mandi history is ingested (the same backtesting discipline the existing ranking engine already used to reach $\rho = 0.884$). **Output:** a ranked list of actions with the top choice and its reasoning (which dimension drove the recommendation) shown, not just a number. **Validation:** backtest — for past dates where the actual future price is now known, check whether the engine’s top-ranked action would have been the better one more often than a naive “always sell immediately” baseline.

Buyer/FPO matching

Inputs: commodity, grade tier, volume range, location, (once available) buyer’s on-chain trust score. **Logic:** hard filters first (same commodity, compatible grade tier, volume within range, location within radius), then a weighted similarity score across the remaining soft factors, ranked descending. **Output:** a ranked shortlist of candidate counterparties, not a single forced match. **Validation:** manual spot-checks against the pilot seed dataset now; once real trades happen, graduate to precision@ k against actual accepted matches.

Trust/reputation score

Inputs: completed, disputed, cancelled, delivered, inspected, on-time, quality-passed trade counts — exactly the fields `TradeLedger.sol` already aggregates on-chain. **Logic:** a transparent weighted formula (e.g. completion rate, on-time rate and quality-pass rate combined, disputes penalised) that anyone can recompute by hand from the public trade history. **Output:** a single score plus the component rates that produced it, always shown together — never the score alone. **Validation:** the formula must be independently reproducible from on-chain data by a third party; that reproducibility *is* the test, not a held-out accuracy number.

5 Build order

1. Formula-based components first (ranking, matching, trust score) — fastest to build, zero training data required, and they unblock the rest of the product flow immediately.
2. Forecaster (Tier: classical ML) — needs real ingested mandi data first (see the dataset landscape doc), then a day or two of feature engineering and baseline comparison.
3. Quality grading Tier 0 (OCR/certificate check) — reuse existing document-intelligence patterns, low effort.
4. Quality grading Tier 1 (classical CV + simple classifier) — start photographing and labelling samples as early as possible; this is the genuine bottleneck in the whole plan, not the modelling step.
5. Quality grading Tier 2 (CNN) — attempt only if Tier 1 works and time plus labelled images allow.

The point of this document

Two components get real modelling effort because the problem genuinely needs statistical learning (forecasting) or visual pattern recognition (grading). Three components deliberately don't get a trained model, because a transparent formula is both more honest about what data you actually have and more defensible in front of a judge who asks "how does this work" and expects a real answer, not "the model learned it."